FPT UNIVERSITY ● FACULTY OF INFORMATION TECHNOLOGY

FINAL CAPSTONE DEFENSE & PROJECT PRESENTATION

OMNICAST

Enterprise Media & Broadcast Intelligence Network

PART 1: PRN232 | Back-End Engineering & Web Portal **PART 2: PRM393** | Mobile Application (Flutter & Dart)

✓ 142,897 Physical LOC ✓ 482/482 Tests PASS (100%) ✓ Live Production on Vercel & Supabase

Author: **OmniCast Engineering Team** ● Academic Semester: **Fall 2026 (FA26)**

October 28, 2026

Executive Summary — The Industry Challenge & The OmniCast Solution

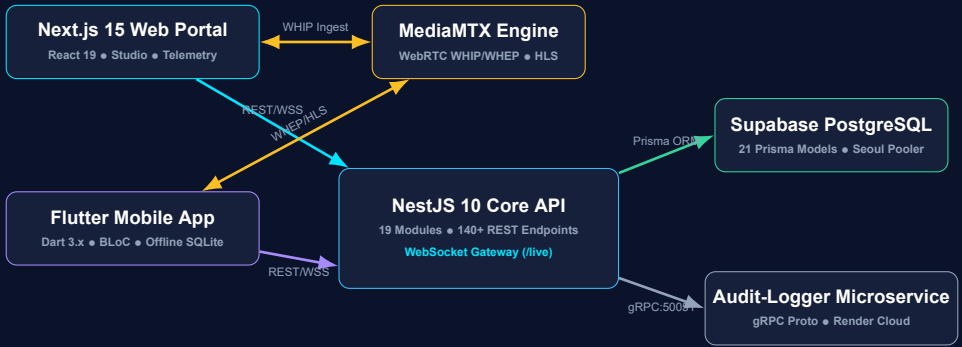
THE PROBLEM: BROADCAST COLLISION & FRAGMENTATION

- **Airtime Collisions:** Overlapping broadcast events cause stream disruptions and schedule inconsistencies.
- **24/7 EPG Dead Air:** Linear TV grids leave empty slots if content runs short, resulting in blank viewer screens.
- **Manual Content Curation:** Broadcasters manually tag, rate, and check compliance across high-volume video catalogs.
- **Rigid Mobile Experiences:** Mobile players freeze without network connection, discarding pending user actions.

THE OMNICAST SOLUTION: UNIFIED HYBRID ECOSYSTEM

- **Mathematical Collision Engine:** Single SQL interval query prevents airtime overlaps at database transaction level.
- **Seeded 24/7 EPG Auto-Filler:** Fills schedule gaps deterministically with reproducible, channel-specific fallback content.
- **AI Broadcast Curator:** LLM-driven structured metadata curation with Zod schema validation and heuristic fallback.
- **Offline-First Mobile Client:** SQLite caching with pending mutation queue and low-latency WebRTC/HLS streaming.

End-to-End System Architecture — Dual-Platform Unified Topology



A unified monorepo architecture bridging broadcast automation, low-latency distribution, and dual-client interaction.

MODULE 1 OF 2

PART 1: PRN232

Building Cross-Platform Back-End Application

NestJS 10 ● Next.js 15 ● PostgreSQL Supabase ● gRPC Microservices ● AI Curator

Backend Architecture & Security — NestJS 10 Modular Core & Three-Tier RBAC

MODULAR DOMAIN DESIGN (19 MODULES)

- **Core Business Modules:** `Channels`, `Programs`, `Epg`, `Live`, `Recordings`, `Billing`.
- **High-Volume Endpoints:** 140+ REST endpoints adhering to strict OpenAPI 3.0 / Swagger specifications.
- **Database Persistence:** 21 Prisma models on Supabase PostgreSQL with connection pooling.
- **Global Validation:** ValidationPipe with class-validator & class-transformer for strict input sanitization.

SECURITY PIPELINE & RBAC MATRIX

- **Authentication Guard:** Dual-token JWT (1h Access Token + 90d Refresh Token rotation).
- **Role-Based Access Control (RBAC):**
  - `ADMIN`: Full governance, user control, audit telemetry.
  - `STAFF`: Broadcaster studio, AI curation, schedule management.
  - `VIEWER`: Live playback, interactive chat, donations, watchlist.
- **Defense in Depth:** Helmet security headers, CORS origin whitelisting, Bcrypt (12 salt rounds).

Algorithmic Contribution 1: Collision Prevention — Mathematical Airtime Overlap

Detection via Raw SQL Interval

THE INTERVAL COLLISION PRINCIPLE

Two broadcasts *A* and *B* collide if and only if:

$$\max(\text{start}_A, \text{start}_B) < \min(\text{end}_A, \text{end}_B)$$

Key Engineering Breakthrough:

- In live broadcasting, an active event's `endTime` may be `NULL` (broadcast ongoing).
- We compute **effective end time** on-the-fly using `COALESCE(p."endTime", p."startTime" + INTERVAL '2 hours')`.
- Executed via atomic SQL raw query inside a Prisma transaction to eliminate race conditions under concurrent bookings.

ATOMIC SQL IMPLEMENTATION

```
SELECT id FROM "Program" p
WHERE p."channelId" = $1
AND p."id" != $2
AND p."startTime" < $effectiveEnd
AND COALESCE(p."endTime",
p."startTime" + INTERVAL '2h')
> $effectiveStart
LIMIT 1;
```

● Validated by `programs.conflict.spec.ts` with 14 boundary test cases (100% PASS).

Algorithmic Contribution 2: 24/7 EPG Auto-Filler — Deterministic Seeded Fallback for Zero Dead-Air Guarantee

THE 24/7 BROADCAST GRID CHALLENGE

- Linear TV channels require an uninterrupted 24-hour timeline (00:00:00 to 23:59:59).
- Original programming typically covers only 2–6 hours per day.
- **Requirement:** Fill schedule holes automatically without user intervention, ensuring exact duration boundaries and non-repeating content.

THE SEEDED REPRODUCIBILITY ENGINE

- **Seeded PRNG Function:**
  Seed $=$ CRC32(ChannelId) $\oplus$ EpochDay.
- **Deterministic Output:** Every client querying the same channel and date receives the identical generated grid.
- **Proportional Time Fitting:** Auto-adjusts filler programs to match exact gap bounds without clipping or drifting.
- **Zero Storage Overhead:** Generated on-the-fly and cached via in-memory cache, saving database space.

Algorithmic Contribution 3: Enterprise AI Curator — LLM Structured Curation with Zod Schema Validation



STRICT SCHEMA ENFORCEMENT

- Validates tags, target demographics, age ratings (G, PG, PG-13, R).
- Generates 3 catchy promotional headlines and EPG summaries.

PRODUCTION RELIABILITY & PROVENANCE

- Zero downtime: In case of API quota exhaustion, heuristic fallback engages.
- Full version history recorded: Every curation run tracks previous vs new tags.

Microservices & Streaming Ingest — High-Speed Audit Logging & MediaMTX WebRTC Pipeline

gRPC AUDIT-LOGGER MICROSERVICE

- **Technology:** NestJS Microservice + Protocol Buffers v3 (`audit.proto`).
- **Transport:** High-performance binary HTTP/2 transport over port `50051`.
- **Zero Main-Thread Blocking:** Asynchronously logs critical events:
  - Program schedule additions and cancellations.
  - Admin user role escalations.
  - AI curation executions and manual overrides.
- **Cloud Deployment:** Render Cloud Container.

MEDIAMTX REALTIME STREAMING NODE

- **Ingest Protocol (WHIP):** WebRTC HTTP Ingestion Protocol allows web studio browsers to broadcast directly from webcams.
- **Distribution Protocol (WHEP & HLS):** Sub-500ms ultra-low latency WebRTC streaming with HLS fallback for constrained networks.
- **Interactive WebSocket Gateway:**
  - Real-time viewer counter (CCU calculation).
  - Floating emoji reactions broadcasting.
  - Live chat room pub/sub with message moderation.

Web Portal Architecture — Next.js 15 App Router, React 19 & Cinema Dark UI

MODERN NEXT.JS 15 APP ROUTER

- **33 Application Routes:** Clean division into public portal, authenticated studio, and admin console.
- **Server Component Performance:** Zero client bundle weight for static metadata, EPG grids, and SEO indexing.
- **Optimized Client Hydration:** Dynamic interactive widgets only where necessary (OmniVideoPlayer, LiveChat).

KEY USER FACING MODULES

- **Broadcast Studio (`/studio`):** Ingests webcam stream, manages active live events, and edits program schedules.
- **Live Curator Workspace (`/studio/curator`):** Interactive AI inspection, tagging overrides, and history comparison.
- **Admin Telemetry (`/admin`):** Real-time spline curves monitoring CCU, CDN bandwidth, and audit trails.

PRN232 Verification & Testing — 391 Automated Tests Across Back-End & Web Portal

BACK-END AUTOMATED TEST SUITE (163 TESTS)

- **Framework:** Node.js native test runner (`node:test`).
- **Core Coverage:**
  - Airtime collision interval edge cases (14 tests).
  - Seeded EPG generator determinism (12 tests).
  - JWT authentication & RBAC guard enforcement (38 tests).
  - AI Curator fallback and validation error handling (19 tests).
- **Status:** 163/163 PASS (100% Success Rate).

WEB PORTAL & E2E TESTING (228+8 TESTS)

- **Component Unit Testing:** 228 Vitest tests validating UI components, forms, and client API states.
- **End-to-End Browser Testing:** 8 comprehensive Playwright E2E scenarios with verified screenshots:
  - Authentication flow (Login, Register, Invalid creds).
  - Channel catalog browsing & category filtering.
  - EPG 24-hour timeline rendering and live viewing.
- **Status:** 100% Verified in CI/CD Pipeline.

MODULE 2 OF 2

PART 2: PRM393

Mobile Applications Development

Flutter 3.x ● Dart ● Clean Architecture ● BLoC Pattern ● Offline SQLite ● OmniPlayer

Mobile Architecture & Patterns

— Four-Tier Clean Architecture with 12 BLoC State Machines

FOUR-TIER CLEAN ARCHITECTURE

- **1. Core Layer:** Dependency Injection via `get_it`, HTTP Dio client with JWT interceptors, Theme constants, Network info.
- **2. Data Layer:** Repositories, DTO models with JSON serialization, local SQLite `DatabaseHelper`.
- **3. Logic Layer:** 12 BLoC and Cubit modules managing asynchronous reactive state streams.
- **4. Presentation Layer:** 31 screens, modular UI components, OmniPlayer, GoRouter navigation.

REACTIVE BLOC STATE MANAGEMENT

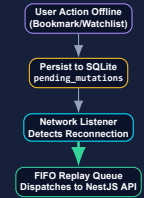
- **Predictable State Machine:** Strict separation between Events, Business Logic, and Immutable UI States.
- **12 Specialized BLoCs:**
  - `AuthBloc`: JWT session, biometric token storage.
  - `ChannelsBloc` & `EpgBloc`: Catalog & grid state.
  - `WatchlistBloc`: Offline bookmarking & sync.
  - `OmniPlayerCubit`: Streaming playback control.
  - `LiveChatCubit`: WebSocket

Mobile Contribution 1: Offline-First SQLite Engine — Local Caching with Asynchronous Mutation Replay Queue

DUAL-LAYER DATA SYNCHRONIZATION

- **Offline Availability:** Users can browse cached channels, read EPG schedules, and check program details with zero connectivity.
- **Local SQLite Schema:**
  - `cached_channels`: Fast cold-start catalog display.
  - `cached_programs`: 24-hour schedule storage.
  - `watchlist`: Saved user favorites.
  - `offline_mutations`: Pending queued mutations.
- **Advance Local Alarms:** Exact 15-minute advance broadcast notification alerts scheduled locally via OS alarm

MUTATION REPLAY QUEUE



Mobile Contribution 2: OmniPlayer Streaming — Multi-Engine WebRTC/HLS with Dynamic In-App LAN Discovery

MULTI-ENGINE OMNIPLAYER

- **Ultra-Low Latency WebRTC:** Directly connects to MediaMTX WHEP endpoint for real-time live video (<500ms).
- **Adaptive HLS Fallback:** Seamlessly falls back to HLS `.m3u8` stream if WebRTC signaling fails.
- **Mobile Controls:** Aspect ratio toggling (16:9, fit, cover), Picture-in-Picture (PiP), and hardware-accelerated rendering.

DYNAMIC LAN HOST DISCOVERY

- **The Mobile Testing Challenge:** Android physical devices cannot reach `localhost:8888` of the development machine.
- **Dynamic In-App Configuration:**
  - Built-in host discovery sheet allows overriding MediaMTX host IP on-the-fly without rebuilding APK.
  - Automatically resolves host IP for zero-fallback camera playback during live grading and testing.

Mobile Contribution 3: Interactive Streaming — Live Chat, Floating Particle Reactions & Virtual Donations

REAL-TIME LIVE CHAT

- Socket.IO client connected to `/live` gateway.
- Fast optimistic message rendering.
- User avatar, role badges (VIP, Mod, Admin).
- System announcement broadcast banners.

FLOATING REACTIONS

- GPU-accelerated canvas particle physics.
- Tap reaction button produces floating hearts, fires, claps.
- Real-time broadcast sync across all connected viewers.

SANDBOX LIVE DONATIONS

- 6 Virtual Gift Tiers (Coffee, Mic, Camera, Trophy, Star, Crown).
- Mock payment modal with instant celebratory confetti.
- 100% risk-free educational sandbox gateway.

Navigation & Verification — GoRouter 31 Screens & 91 Automated Tests (100% Pass)

GOROUTER DECLARATIVE ROUTING

- **31 Distinct Screens:** Spanning Auth, Catalog, EPG, Player, Creator Studio, Watchlist, Settings.
- **Auth Guard Redirects:** Automatic redirection if JWT token expires or required roles are missing.
- **Cinema Dark Aesthetic:** High-contrast UI optimized for OLED screens, haptic feedback on touch gestures.

91 AUTOMATED TESTS (100% PASS)

- **Unit Tests (56 tests):** BLoC state transition testing (`bloc_test`), SQLite repository operations, DTO serialization.
- **Widget Tests (35 tests):** UI component rendering, button interactions, mock network failure handlers.
- **CI/CD Integration:** Tested automatically on every git commit via GitHub Actions.

Cross-Platform Synergy Matrix — Side-by-Side Comparison: PRN232 Back-End vs PRM393 Mobile

Dimension	PRN232 (Back-End & Web)	PRM393 (Mobile App)
Core Framework	NestJS 10 + Next.js 15 App Router	Flutter 3.x + Dart 3
Architecture	Layered Clean Modules + gRPC	4-Tier Clean Architecture + BLoC
Database / Cache	Supabase PostgreSQL (21 models)	SQLite Local (`sqflite`) + SharedPrefs
Streaming Role	Ingest (WHIP), Signaling, EPG	Player (WHEP/HLS), Floating UI
Realtime Engine	Socket.IO Gateway Server (`/live`)	Socket.IO Client + Particle Canvas
Code Volume	**79,568 Physical LOC**	**53,922 Physical LOC**
Automated Tests	**391 Tests** (163 API + 228 Web)	**91 Tests** (Unit + Widgets)
Verification Rate	**100% PASS**	**100% PASS**

TOTAL SYSTEM SCALE: 142,897 Lines of Code ● 482 Automated Tests ● 100% Pass

Live Deployment & Evaluation — Production Cloud Infrastructure & Pre-Seeded Test Credentials

LIVE PRODUCTION ENDPOINTS

- **Web Portal (Vercel Live):** `https://omnicast-fe.vercel.app`
- **Core API Swagger Documentation:** `https://omnicast-api.vercel.app/swagger`
- **Mobile APK Release (v1.2.0):** `github.com/TomOutfit/.../releases/tag/v1.2.0`
- **Official GitHub Repository:** `github.com/TomOutfit/OmniCast_PRN232-PRM393...`

PRE-CONFIGURED EVALUATION ACCOUNTS

Password for all test accounts: `Admin123!`

- **Admin Account (Full RBAC):** `admin@omnicast.tv`
- **Staff Account (Studio Broadcaster):** `staff@omnicast.tv`
- **Viewer Account (Consumer):** `viewer1@omnicast.tv`

Project Milestones & Lessons Learned — Engineering Takeaways from Joint Capstone

Implementation

KEY TECHNICAL ACHIEVEMENTS

- Successfully bridged complex broadcast engineering with cloud-native web and native mobile clients.
- Eliminated airtime overlap risks through atomic SQL interval calculation.
- Solved the 24/7 dead air dilemma using deterministic seeded pseudo-random filling.
- Provided seamless offline resilience for mobile users through SQLite mutation queuing.

LESSONS LEARNED & FUTURE EXTENSIONS

- **gRPC Efficiency:** Binary proto serialization drastically reduces latency compared to conventional REST.
- **BLoC Event Isolation:** Strict event streams prevent UI stuttering during heavy network calls.
- **Future Work:** Distributed MediaMTX clustering across edge nodes, AI auto-dubbing, and adaptive multi-bitrate transcoding.

Thank You for Your Attention!

OmniCast: Enterprise Media & Broadcast Intelligence Network

We are ready for Live Demonstration & Committee Q&A

Web: omnicast-fe.vercel.a **API:** omnicast-api.vercel.app/swag **GitHub:** github.com/TomOutfit/OmniCast

Department of Software Engineering ● FPT University ● Fall 2026